Assignment 2: Cost-Efficient Segmentation under a Compute Budget

Jatin Purbia — B23CH1022
CSL7360 — Computer Vision
Indian Institute of Technology Jodhpur

29 October 2026

Abstract

This project implements Track B using original lightweight fully convolutional networks trained from scratch. It shares one preprocessing and inference implementation across prediction, evaluation and timing, and computes the prescribed metrics manually. Experimental claims are limited to saved evidence. The results table explicitly identifies missing evaluations; authentic personal photographs and their annotations cannot be replaced by synthetic evidence. This document is an implementation report until all required experiments and photographs are present.

1 Introduction

The task is binary salient-object segmentation: assign each pixel to the main object(s) or background without interactive object selection. The objective is to maximize the prescribed Cost-Efficient Segmentation (CES) score subject to a mean single-thread CPU inference budget of 50 milliseconds. Region overlap alone is insufficient: the quality term also rewards accurate boundaries. Three small encoder–decoder designs explore standard, deeper and depthwise separable convolutions. No pretrained model or parameters are used.

2 Dataset and leakage prevention

DUTS contains 10,553 training photographs (DUTS-TR) and 5,019 test photographs (DUTS-TE) [2]. A sorted list of DUTS-TR stems is shuffled using Python's seed-42 random generator; round($0.1N$) IDs form validation. For the official training size this gives 9,498 training and 1,055 validation IDs. The actual manifests, once prepared, are saved and hashed. Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42.

All architecture, loss, threshold, resolution and stopping decisions use only DUTS-TR validation. The experiment runner never loads DUTS-TE. Final evaluation requires a frozen parameter-file hash and records all 5,019 samples individually. Personal photos are testing-only and require 5–10 authentic phone captures covering at least three condition categories. A script can validate files and author attestations; it cannot prove photographic authenticity or scene diversity.

3 Preprocessing

OpenCV reads BGR uint8 photographs. Every original photo is directly resized to 256×256 using `INTER_AREA`; aspect ratio is intentionally squashed. Grayscale masks use `INTER_NEAREST` and

are binarized at intensity 128 inclusive. The network receives RGB float32 tensors $[B, 3, H, W]$ in $[0, 1]$, with no mean/std normalization. Training augmentation jointly flips image/mask horizontally with probability 0.5 and applies brightness/contrast multipliers sampled uniformly in $[0.9, 1.1]$ to images only. Validation and test preprocessing are deterministic.

The timed pipeline starts after the original resize. It includes BGR-to-RGB conversion, tensor creation, normalization, optional bilinear downscaling, forward inference, bilinear upsampling of logits to 256×256, sigmoid, thresholding and conversion to a binary uint8 array. Masks on disk are multiplied by 255 after timing.

4 Proposed architectures

The default three-stage Tiny FCN uses widths $(b, 2b, 4b)$. Each encoder block is Conv–BatchNorm–ReLU; max pooling halves spatial size between blocks. The decoder bilinearly upsamples to each skip's size, concatenates encoder features and applies a new convolution block. A 1×1 head produces a single logit channel. The U-Net candidate adds a second convolution in each encoder stage. The separable candidate replaces each block with a depthwise spatial convolution followed by a pointwise projection, each normalized and activated. These designs were written using permitted PyTorch layers rather than importing a model architecture.

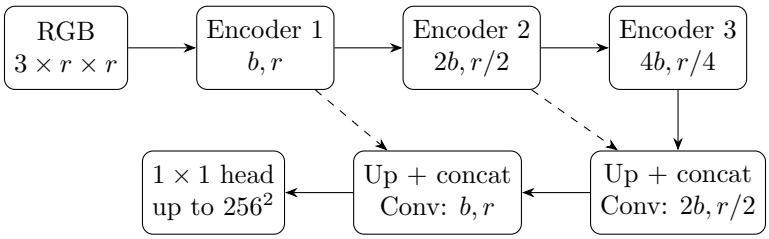


Figure 1: Actual three-stage Tiny FCN topology. r is internal resolution; b is base width. Dashed arrows are concatenation skips. The chosen configuration and parameter count must be read from the frozen run, not inferred from this generic diagram.

Depth, base width, decoder width, kernel size, skip usage, and bilinear versus transpose upsampling are configurable. Untrained parameter/latency diagnostics are recorded separately from quality experiments. Fewer parameters do not establish lower measured latency. Final architecture selection is pending measured validation comparisons.

5 Mathematical formulation

For input channel c, output channel o and spatial offsets (u, v), convolution is

$$z_{oij} = a_o + \sum_c \sum_u \sum_v W_{ocuv} x_{c,i+u,j+v}. \tag{1}$$

Here W contains learned filters and a is a bias (omitted in normalized blocks). For logit z_i, probability p_i and binary label y_i,

$$p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}, \qquad \hat{y}_i = \mathbf{1}[p_i \geq \tau], \tag{2}$$

$$L_{\text{BCE}} = -\frac{1}{N}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \tag{3}$$

$$L_{\text{Dice}} = 1 - \frac{2\sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + \epsilon}, \qquad \epsilon = 10^{-6}, \tag{4}$$

$$L = L_{\text{BCE}} + \beta L_{\text{Dice}}. \tag{5}$$

BCE is computed through the stable logits formulation. Dice is computed separately for each image and averaged. The default decision threshold is $\tau = 0.5$; any change must use validation only. The mandatory baseline has $\beta = 0$.

Aggregate all pixels in the evaluated dataset into

$$C = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}, \qquad TP = \sum_i \mathbf{1}[\hat{y}_i = 1, y_i = 1], \tag{6}$$

with analogous definitions for true negatives, false positives and false negatives. Rows denote true class; columns denote predicted class. Then

$$\text{IoU}_{fg} = \frac{TP}{TP+FP+FN}, \qquad \text{IoU}_{bg} = \frac{TN}{TN+FP+FN}, \tag{7}$$

$$\text{mIoU} = \tfrac{1}{2}(\text{IoU}_{fg} + \text{IoU}_{bg}). \tag{8}$$

We assign an absent class (zero union) IoU of 1, representing correct absence. This is an explicit implementation convention because the PDF does not define that degeneracy. Per-image IoUs are never averaged to form the dataset score.

For foreground set M and four-connected neighborhood $\mathcal{N}_4$,

$$\partial M = \{i \in M : \exists j \in \mathcal{N}_4(i),\ j \notin M\}. \tag{9}$$

Pixels outside the image are background. Let $B_p = \partial\hat{M}$, $B_t = \partial M$, and $D_2(B)$ be pixels within a centered 5×5 window of B. Matching tolerance is Chebyshev distance two, including diagonal offsets $(2,2)$. For one image,

$$P = \frac{|B_p \cap D_2(B_t)|}{|B_p|}, \qquad R = \frac{|B_t \cap D_2(B_p)|}{|B_t|}, \tag{10}$$

$$F_1 = \begin{cases} 2PR/(P+R) & P+R>0, \\ 0 & \text{otherwise}, \end{cases} \qquad BF1 = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}. \tag{11}$$

Both empty boundaries yield $F_1 = 1$; exactly one empty boundary yields $F_1 = 0$. Four directional tensor slices extract the boundary, and permitted `max_pool2d` performs neighborhood matching without library morphology or per-pixel Python loops. Matching is many-to-many, as specified, rather than bipartite assignment.

For all quality metrics on the $[0,1]$ scale,

$$Q = 0.70\,\text{mIoU} + 0.30 BF1, \tag{12}$$

$$CES = \begin{cases} Q\left[1 - 0.20\frac{T_{CPU}}{50} - 0.10\min\left(1, \frac{S_{disk}}{4194304}\right)\right], & T_{CPU} \leq 50, \\ 0, & T_{CPU} > 50. \end{cases} \tag{13}$$

T_{CPU} is in milliseconds and S_{disk} is the actual parameter-file size in bytes. For $Q = .7$, $T = 20$ and $S = 524288$, the exact result is .63525, consistent with the PDF's rounded .635.

6 Training methodology

The baseline configuration uses AdamW, learning rate 10^{-3}, weight decay 10^{-4}, cosine scheduling over 40 epochs, batch size 8, patience 8 and gradient norm clipping at 5. The explicitly limited CPU pilot configuration uses two epochs, batch size 16 and smaller widths at resolution 128. These are different experiment budgets and must be identified when interpreting results. Every run saves its complete configuration, package versions, manifest hashes, best epoch, best validation metrics and per-epoch CSV logs. Best epoch selection maximizes validation Q within each training run. Across candidates, final selection maximizes measured validation CES. Training checkpoints include optimizer and scheduler states; inference exports retain all learned weights and batch-normalization buffers.

7 Computational efficiency and timing

The encoder operates at a configurable internal resolution; the full output remains 256×256. Standard blocks prioritize simple CPU kernels, while separable blocks reduce arithmetic at the possible cost of additional operator overhead. Single-thread execution is configured before NumPy, OpenCV or PyTorch imports through four required environment variables, followed by their thread-setting APIs. Inference uses CPU tensors, `eval()` and `inference_mode()`. No inference subprocess, data-loader worker or alternate engine is used.

One full warmup is excluded. For official timing, each DUTS-TE photo is loaded and resized outside the timer. The exact shared inference function is wrapped by `time.process_time()`. The sum of all 5,019 durations divided by 5,019 gives T_{CPU}. Metrics and file exports occur outside timing. The prediction CLI instead uses its output-producing pass as warmup, then averages 20 additional full-pipeline runs. Size uses `os.path.getsize`; plain `torch.save` is not post-compressed.

8 Experimental comparison

The planned comparison isolates architecture, BCE versus BCE+Dice, and internal resolution. `results/model_comparison.csv` contains only measured DUTS-TR validation scores. The selected row is saved with its checkpoint provenance before either final test set is opened. Synthetic fixtures verify software behavior; synthetic latency diagnostics do not establish DUTS segmentation quality or official timing compliance. Validation CES is an estimate for selection, while only the complete DUTS-TE CES is official.

9 Results

Set	mIoU	BF1	Q	CPU ms	Bytes	CES	N
Validation	Not evaluated; prerequisite evidence pending						
DUTS-TE	Not evaluated; prerequisite evidence pending						
Captured photos	Not evaluated; prerequisite evidence pending						

Table 1: Evidence-derived results. Each row's time is measured on that set; storage is shared for the same checkpoint. Validation CES is an estimate; DUTS-TE CES is official only after all 5,019 images are evaluated. Missing evidence is reported as not evaluated.

CPU and saved split information: Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42. Timing compliance must be assessed from the complete DUTS-TE row, not a short synthetic benchmark or an individual prediction.

10 Captured-photo results

No genuine captured photographs or annotations have been supplied. Evaluation and mandatory overlays remain incomplete. All captures must appear, including unsuccessful cases. Annotation records document objects, lighting, distance, clutter, occlusion and manual annotation method. Ground truth and predictions are shown with the same green foreground and opacity 0.45 over the corresponding image.

11 Failure analysis

At least two genuine failure explanations remain pending evaluated-image review. The visualization utility ranks actual observed examples by per-image quality to suggest candidates; a low score alone does not establish its cause. No lighting, occlusion, clutter or small-object failure is claimed without inspecting the relevant photo and mask.

12 Conclusion and limitations

The implementation provides an auditable route from deterministic data preparation to trained parameter export and shared CPU evaluation. Architecture complexity is justified only by measured validation performance. Final segmentation quality, the full CPU budget, captured-photo generalization and submission completeness depend on the evidence in the results table. Pending entries prevent a submission-ready claim.

13 Reproducibility

Run these PowerShell commands from a fresh project copy. Backticks continue a command on the next line. Training outputs and final-test evidence must not overwrite an existing run. Download DUTS-TE only after validation design choices and freezing are complete. The complete selected training configuration is saved in `configs/final.json`; reproduce it in a new run directory. All commands and extra experiment options are documented in the root README and `docs/REPRODUCIBILITY.md`.

```
python -m pip install -r requirements.txt
python -m scripts.verify
python -m scripts.download_duts --split TR
python -m scripts.prepare_dataset --root data/DUTS
python train.py --config configs/final.json `
  --out experiments/reproduce_final
# Actual limited comparison, in an unused output directory:
python -m scripts.run_experiments --config configs/pilot.json `
  --experiments configs/pilot_experiments.json --out experiments/pilot4
# Complete all validation sweeps before this step:
python -m scripts.freeze_model
$tr = 'data/DUTS/DUTS-TR'
python evaluate.py --split validation --images "$tr/DUTS-TR-Image" `
  --masks "$tr/DUTS-TR-Mask" --manifest data/splits/val.txt `
  --out results/validation_metrics.json
python -m scripts.download_duts --split TE
$te = 'data/DUTS/DUTS-TE'
# This is both official quality evaluation and CPU timing:
python benchmark.py --split duts-test --images "$te/DUTS-TE-Image" `
  --masks "$te/DUTS-TE-Mask" --out results/duts_test_metrics.json
python predict.py --image photo.jpg --out mask.png
# Only after authentic capture and manual annotation:
python -m scripts.verify_dataset --root captured --split captured
python evaluate.py --split captured --images captured/images_256 `
  --masks captured/masks_256 --out results/captured_metrics.json
python -m scripts.create_figures overlays `
  --images captured/images_256 --masks captured/masks_256 `
  --out results/captured_figures
python -m scripts.create_figures comparison
python -m scripts.build_report
python -m scripts.audit --submission
python -m scripts.package_submission
```

Prediction loads `weights/final_model.pth` and prints exactly `T_CPU_ms:` and `S_disk_bytes:`. The first pass writes the mask; the next 20 passes are timed. Report building supports pdflatex or an explicitly supplied portable Tectonic executable. Every compiled page requires visual review. Unavailable captured-photo evidence prevents the final audit from passing, even if all software tests pass.

References

[1] CSL7360, *Assignment 2: Cost-Efficient Segmentation under a Compute Budget*, supplied assignment PDF, pp. 1–3.

[2] L. Wang et al., *Learning to Detect Salient Objects with Image-level Supervision*, CVPR, 2017. Dataset: `https://saliencydetection.net/duts/`.